LAPORAN PRAKTIKUM MICROPROCESSOR & INTERFACE

PROGRAM PENJUMLAHAN BCD



PRAKTIKAN

Nama : Elsadanika Maura Sitorus

NIM : 211344013

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2022/2023

KETERANGAN

1. Kelompok : -
2. Judul Laporan : Program Penjumlahan BCD
3. Tanggal Praktikum : 31 Januari 2023
4. Tanggal Pengumpulan : 04 Februari 2023
5. Nama Praktikan : Elsadanika Maura Sitorus
6. Nama Partner : -
7. Nama Dosen Pembimbing : Ferry Satria, BSEE., M. T.
   Rahmawati Hasanah, S. ST., M. T.

DAFTAR ISI

KETERANGAN i
DAFTAR ISI ii

A. Tujuan Praktikum 1
B. Dasar Teori 1
C. Alat dan Komponen yang Digunakan 3
D. Langkah Kerja 4
E. Data Pengamatan 5
F. Analisis Data 9
G. Kesimpulan 12
H. Daftar Pustaka 13

A. Tujuan Praktikum

1. Memahami bahasa Assembly
2. Memahami cara penggunaan simulator EMU 8086
3. Memahami pemanfaatan EMU 8086 simulator untuk menjumlahkan data bilangan BCD menggunakan bahasa pemrograman Assembly.

B. Dasar Teori

1. Perintah Looping, Memory, dan Data Area.

Arahan assembler define digunakan sebagai alokasi ruang penyimpanan dan untuk menginisialisasi satu atau lebih byte.

a) Mengalokasikan Ruang Penyimpanan untuk Data yang Diinisialisasi

Sintaks untuk pernyataan alokasi penyimpanan untuk data yang diinisialisasi adalah sebagai berikut:

```
[nama-variabel] define-directive initial-value [,initial-value]...
```

Nama variabel adalah pengidentifikasi untuk setiap ruang penyimpanan. Assembler mengaitkan nilai offset untuk setiap nama variabel yang ditentukan dalam segmen data. Ada lima bentuk dasar dari direktif define yang ditampilkan pada tabel berikut:

Perintah	Keterangan	Storage Space
DB	Define Byte	1 byte
DW	Define Word	2 bytes
DD	Define Doubleword	4 bytes
DQ	Define Quadword	8 bytes
DT	Define Ten Bytes	10 bytes

b) Perintah Looping

Instruksi untuk transfer perintah selama eksekusi disesuaikan dengan kondisinya, berikut ini adalah instruksi untuk proses looping dalam Bahasa assembly pada 8086:

Instructions	Conditions
JA/JNBE	Jump if above/not below/equal instruction satisfies
JAE/JNB	Jump if above/not below
JBE/JNA	Jump if below/equal/ not above

JC	Jump if carry flag CF = 1
JE/JZ	Jump if equal/zero flag ZF = 1
JG/JNLE	Jump if greater/not less than/equal
JGE/JNL	Jump if greater than/equal/not less
JL/JNGE	Jump if less than/not greater than/equal
JLE/JNG	Jump if less than/equal/if not greater
JNC	Jump if no carry flag (CF = 0)
JNE/JNZ	Jump if not equal/zero flag ZF = 0
JNO	Jump if no overflow flag OF = 0
JNP/JPO	Jump if not parity/parity odd PF = 0
JNS	Jump if not sign SF = 0
JO	Jump if overflow flag OF = 1
JP/JPE	Jump if parity/parity even PF = 1
JS	Jump if sign flag SF = 1

2. Kondisi Overflow

Overflow merupakan kondisi ketika hasil operasi mengalami over capacity terhadap register atau memory areanya. Overflow dilihat dari range data. Berikut ini adalah range data 8 bit:

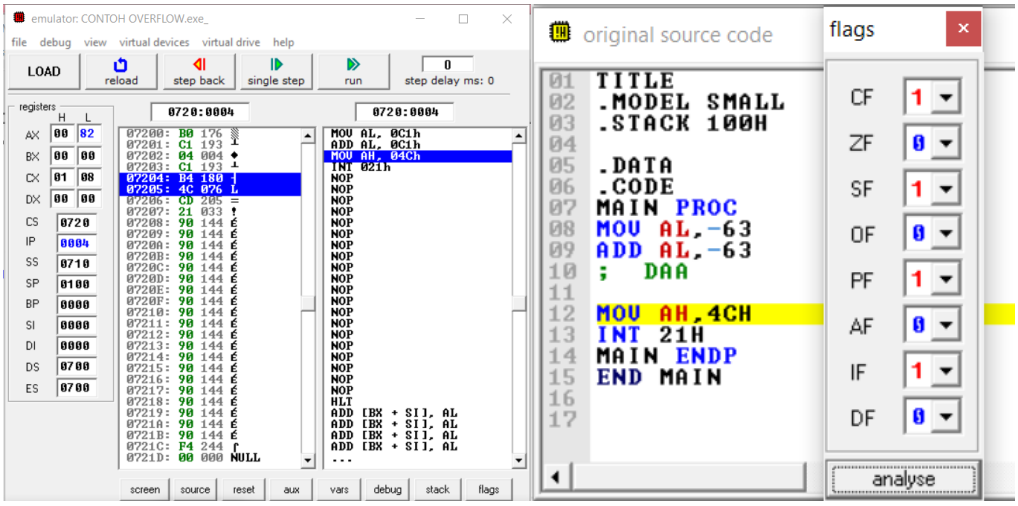
Data 8 bit tidak bertanda	Data 8 bit bertanda
0 - 255	(-128) – (+127)

Dalam penjumlahan decimal, overflow diketahui ketika hasil penjumlahan melebihi 8 bit, sehingga satu memory area tidak dapat menampungnya dan memerlukan area lebih. Berikut adalah contoh penjumlahan decimal dengan kondisi overflow.

Data 1	Data 2	Hasil	Overflow
79	02	81	00
79	79	58	01

Untuk mengetahui suatu penjumlahan decimal mengalami overflow maka, bilangan diubah ke bentuk biner komplemen 2 terlebih dahulu lalu diubah ke heksadesimal pada simulator. Berikut ini adalah contoh penjumlahan bilangan decimal yang tidak mengalami overflow pada simulator.

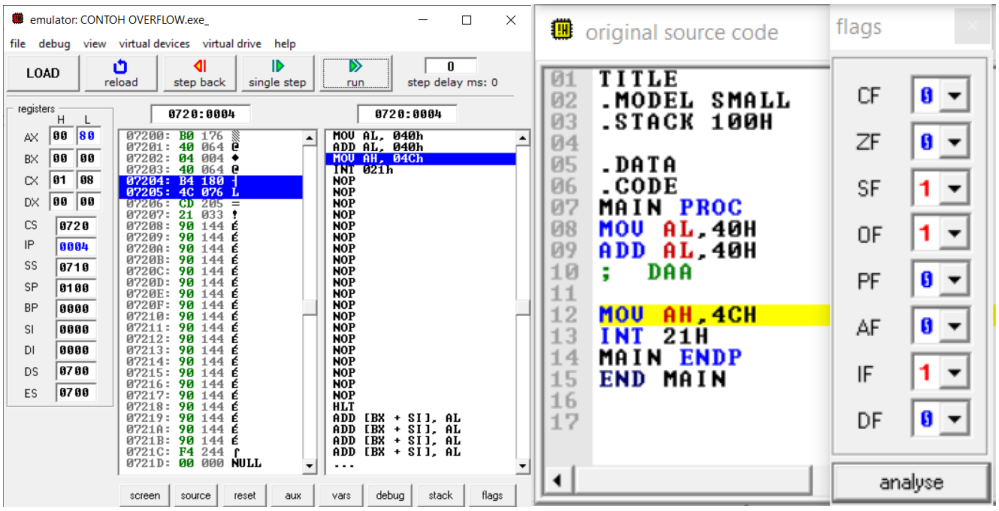
$-63_{(10)} = 1\ 0111111_{(2SM)} \rightarrow 1\ 1000000_{(K'1)} \rightarrow 1\ 1000001_{(K'2)} \rightarrow C1_{(16)}$
(-63) + (-63) = (-126)



Berikut ini adalah contoh penjumlahan bilangan hexadecimal yang mengalami overflow pada simulator.

$40H_{(16)} = 0\ 1000000_{(2)} \rightarrow 64_{(10)}$

64 + 64 = 128



C. Alat dan Komponen yang Digunakan

- Laptop/Personal Computer
- Simulator EMU 8086

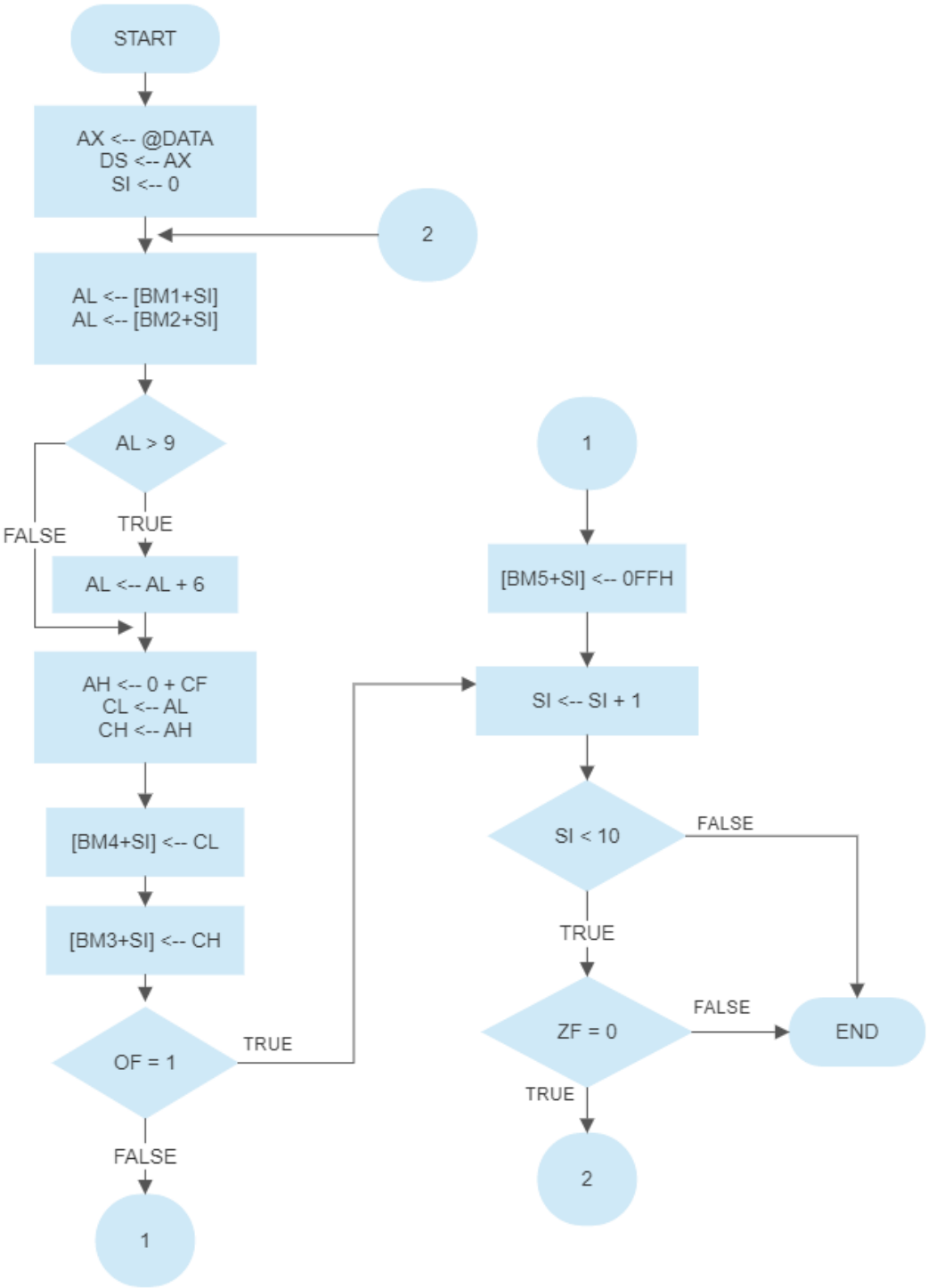
D. Langkah Kerja

1. Buat diagram alir dan program untuk menjumlahkan 10 pasang data decimal 2 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3. Beri tanda ketika terjadi overflow.
   BM1 DB 93H,77H,63H,94H,32H,44H,17H,47H,42H,75H
   BM2 DB 37H,49H,63H,81H,93H,27H,43H,23H,35H,12H
   BM3 DB 10 DUP(0)

2. Buat diagram alir dan program untuk menjumlahkan 10 pasang data decimal 4 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3. Beri tanda ketika terjadi overflow.
   BM1 DW 1693H,7887H,6381H,94H,32H,4423H,1987H,4700H,4902H,7935H
   BM2 DW 3437H,4996H,63H,8109H,9443H,2762H,4023H,6723H,3225H,1298H
   BM3 DW 10 DUP(0)
3. Buat diagram alir dan program untuk menjumlahkan data decimal 8 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3. Beri tanda ketika terjadi overflow.
   BM1 78903456H,89674321H,87654321H,0H,22117755H
   BM2 12345678H,98766550H,55667723H,12345678H,62117755H
   BM3 0,0,0,0,0
4. Buat diagram alir dan program untuk menjumlahkan 4 pasang data decimal 9 digit. Data input dan outputnya tentukan sendiri.

E. Data Pengamatan

1. Flowchart



Program

```
TITLE PENJ 10 BIL DECIMAL DAN CEK OF
.MODEL SMALL
.STACK 100H

.DATA
BM1 DB 93H,77H,63H,94H,32H,44H,17H,47H,42H,75H
BM2 DB 37H,49H,63H,81H,93H,27H,43H,23H,35H,12H
BM3 DB 10 DUP(0)
BM4 DB 10 DUP(0)
BM5 DB 10 DUP(0)
;HASIL : 130,126,126,175,125,71,60,70,77,87

;.........................................
;PROGRAMMER : ELSADANIKA MAURA SITORUS
;NIM-KELAS  : 211344013 - 2NK
;TANGGAL    : 01/02/23
;.........................................

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX
    MOV SI,0

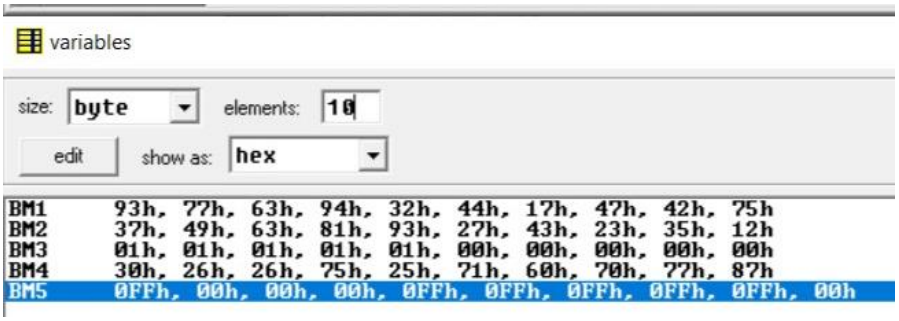
L1: MOV AL,[BM1+SI]
    ADD AL,[BM2+SI]
    PUSHF          ;MENYIMPAN FLAG PADA REGISTER FLAG
    DAA
    MOV AH,0
    ADC AH,0
    MOV CL,AL
    MOV CH,AH
    MOV [BM4+SI],CL
    MOV [BM3+SI],CH
    POPF           ;MENGAMBIL FLAG REGISTER DARI STACK

    JO L2
    MOV [BM5+SI],0FFH

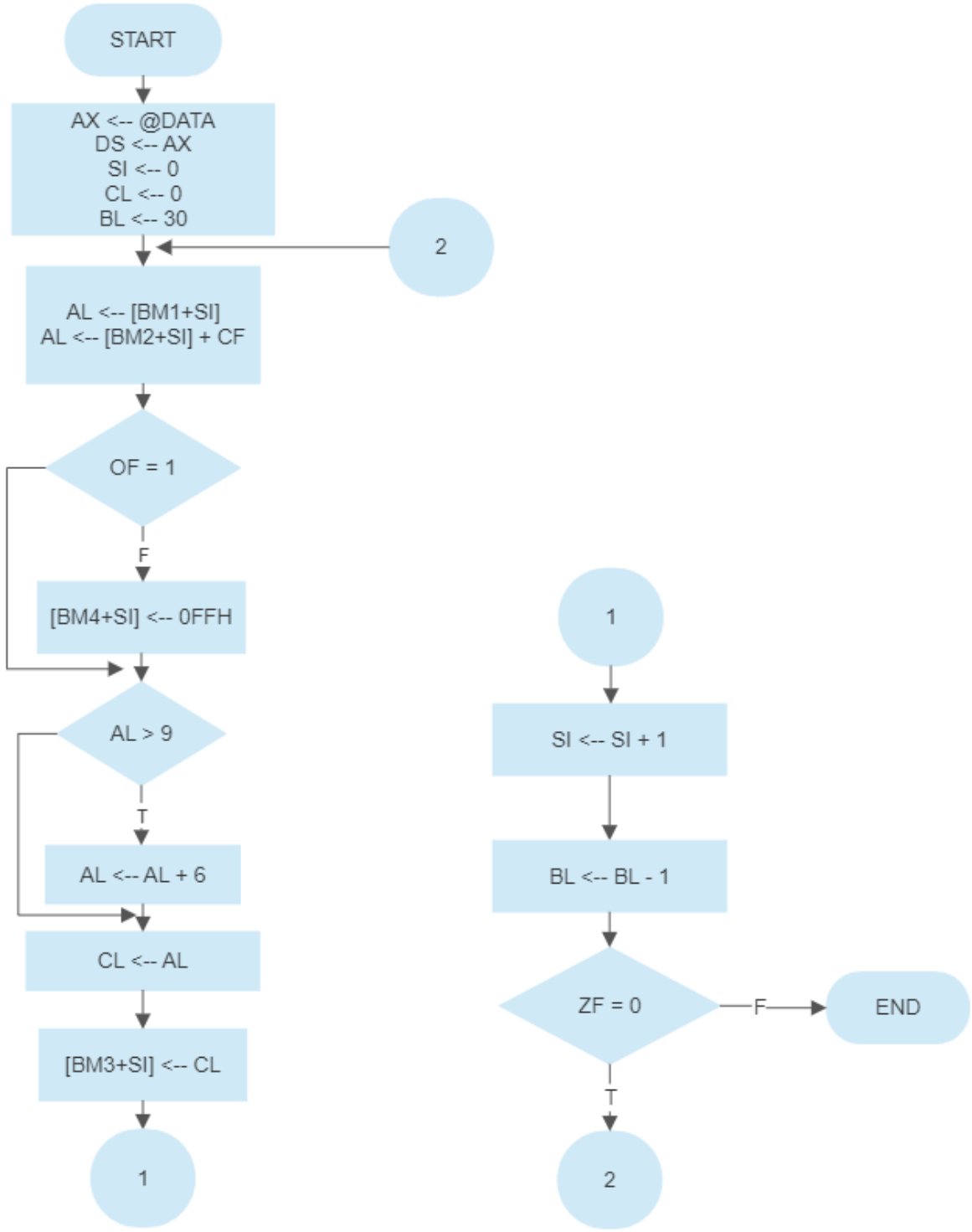
L2: INC SI
    CMP SI,10      ;COMPARE SI
    CLC
    JNZ L1

    MOV AH,4CH
    INT 21H
    MAIN ENDP
END MAIN
```

Output



2. Flowchart


START
AX <-- @DATA
DS <-- AX
SI <-- 0
CL <-- 0
BL <-- 30
2
AL <-- [BM1+SI]
AL <-- [BM2+SI] + CF
OF = 1
F
[BM4+SI] <-- 0FFH
AL > 9
T
AL <-- AL + 6
CL <-- AL
[BM3+SI] <-- CL
1
1
SI <-- SI + 1
BL <-- BL - 1
ZF = 0
F
END
T
2

Program

```
TITLE PENJ 10 BIL DECIMAL 4 DIGIT DAN CEK OF
.MODEL SMALL
.STACK 100H

.DATA
BM1 DB 93H,16H,00H,87H,78H,00H,81H,63H,00H,94H,00H,00H,32H,00H,00H,23H,44H,00H,87H,19H,00H,00H,47H,00H,02H,49H,00H,35H,79H,00H
BM2 DB 37H,34H,00H,96H,49H,00H,63H,00H,00H,09H,81H,00H,43H,94H,00H,62H,27H,00H,23H,40H,00H,23H,67H,00H,25H,32H,00H,98H,12H,00H
BM3 DB 30 DUP(0)
BM4 DB 30 DUP(0)
;HASIL : 5130,12883,6444,8203,9475,7185,6010,11423,8127,9223

;.....................................
;PROGRAMMER : ELSADANIKA MAURA SITORUS
;NIM-KELAS  : 211344013 - 2NK
;TANGGAL    : 01/02/23
;.....................................

.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX
    MOV SI,0
    MOV CL,0
    MOV BL,30

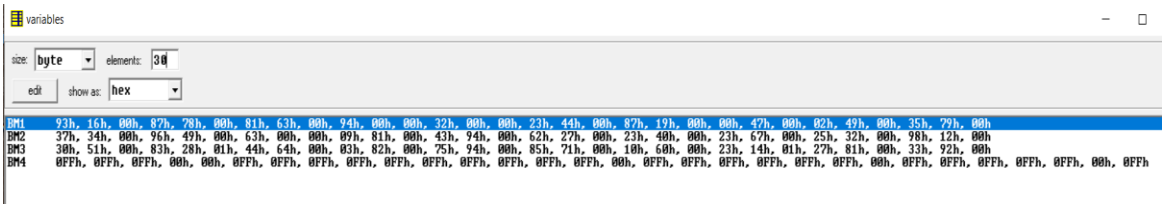
L1: MOV AL,[BM1+SI]
    ADC AL,[BM2+SI]
    JO L2
    MOV [BM4+SI],0FFH

L2: DAA
    MOV CL,AL
    MOV [BM3+SI],CL

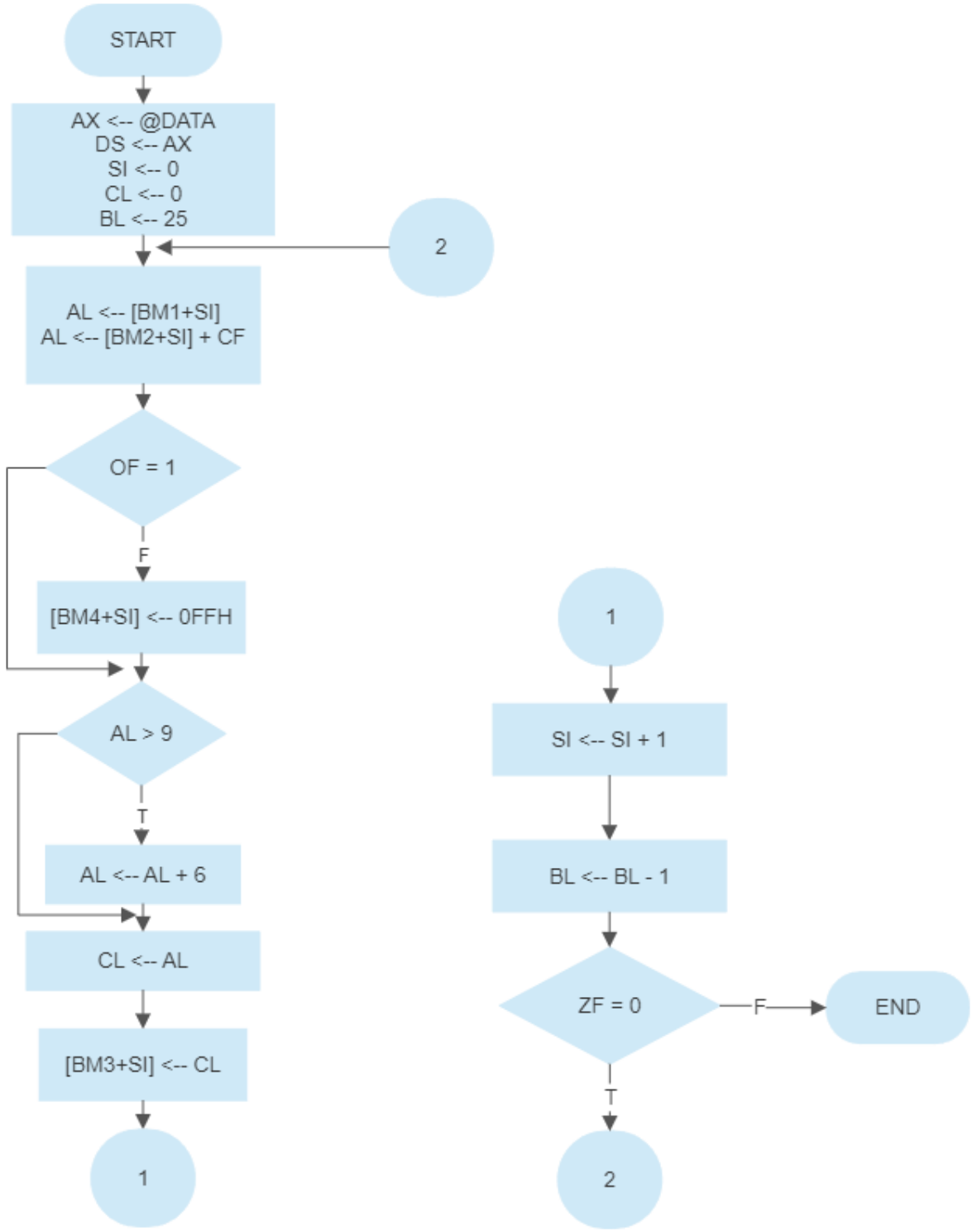
    INC SI
    DEC BL
    JNZ L1

    MOV AH,4CH
    INT 21H
    MAIN ENDP
END MAIN
```

Output



3. Flowchart


START
AX <-- @DATA
DS <-- AX
SI <-- 0
CL <-- 0
BL <-- 25
2
AL <-- [BM1+SI]
AL <-- [BM2+SI] + CF
OF = 1
F
[BM4+SI] <-- 0FFH
AL > 9
T
AL <-- AL + 6
CL <-- AL
[BM3+SI] <-- CL
1
1
SI <-- SI + 1
BL <-- BL - 1
ZF = 0
F
END
T
2

Program

```
TITLE PENJ DECIMAL 8 DIGIT DAN CEK OF
.MODEL SMALL
.STACK 100H

.DATA
BM1 DB 56H,34H,90H,78H,00H,21H,43H,67H,89H,00H,21H,43H,65H,87H,00H,00H,00H,00H,00H,00H,55H,77H,11H,22H,00H
BM2 DB 78H,56H,34H,12H,00H,50H,65H,76H,98H,00H,23H,77H,66H,55H,00H,78H,56H,34H,12H,00H,55H,77H,11H,62H,00H
BM3 DB 25 DUP(0)
BM4 DB 25 DUP(0)

;HASIL : 91249134, 188440871, 143322044, 12345678, 84235510

;.........................................
;PROGRAMMER : ELSADANIKA MAURA SITORUS
;NIM-KELAS  : 211344013 - 2NK
;TANGGAL    : 01/02/23
;.........................................

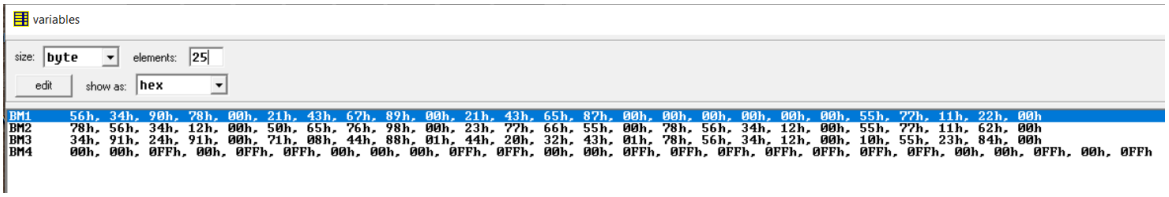
.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV SI,0
     MOV CL,0
     MOV BL,25

L1: MOV AL,[BM1+SI]
    ADC AL,[BM2+SI]
    JO L2
    MOV [BM4+SI],0FFH

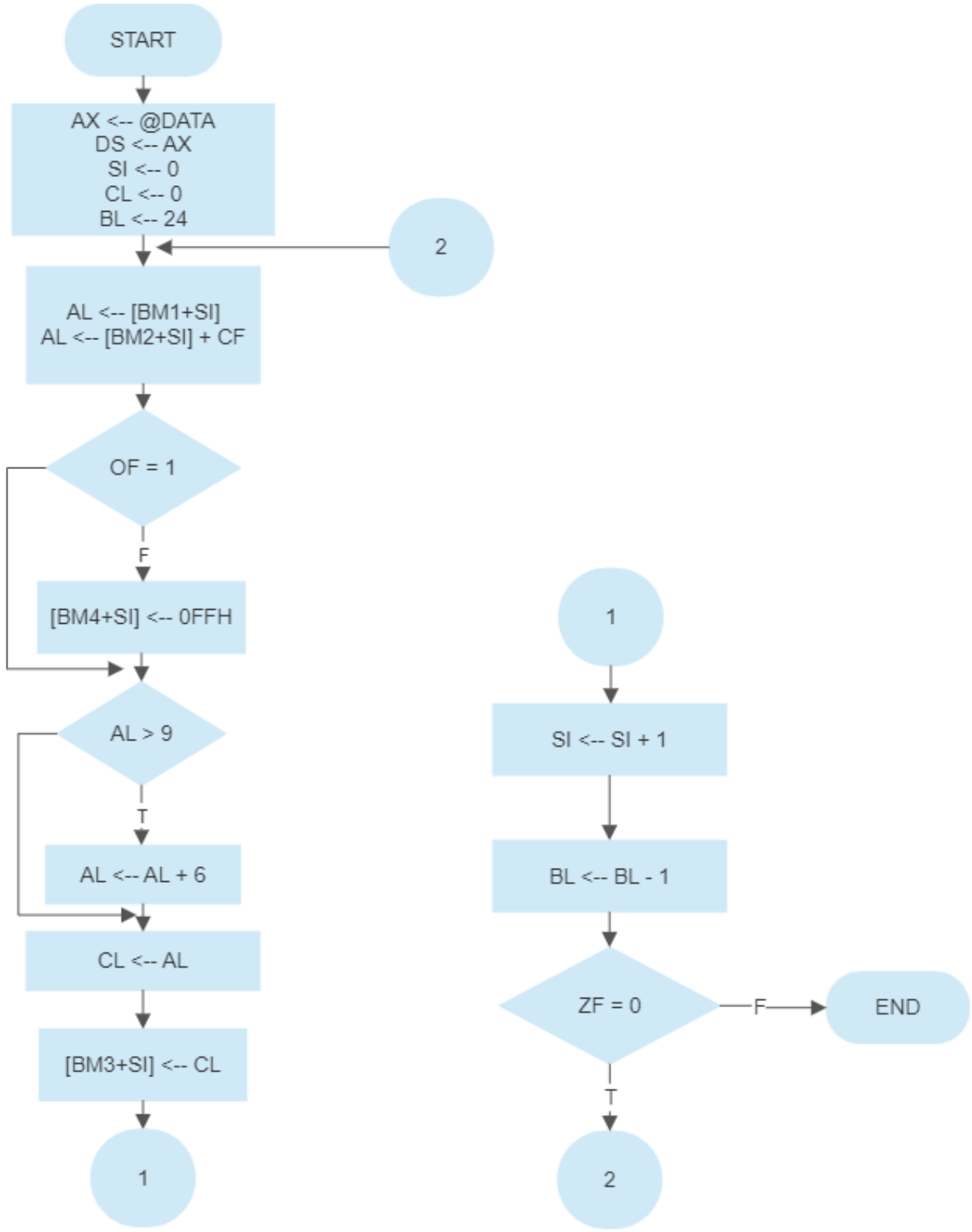
L2: DAA
    MOV CL,AL
    MOV [BM3+SI],CL
    INC SI
    DEC BL
    JNZ L1

    MOV AH,4CH
    INT 21H
    MAIN ENDP
END MAIN
```

Output



4. Flowchart


START
AX <-- @DATA
DS <-- AX
SI <-- 0
CL <-- 0
BL <-- 24
2
AL <-- [BM1+SI]
AL <-- [BM2+SI] + CF
OF = 1
F
[BM4+SI] <-- 0FFH
AL > 9
T
AL <-- AL + 6
CL <-- AL
[BM3+SI] <-- CL
1
1
SI <-- SI + 1
BL <-- BL - 1
ZF = 0
F
END
T
2

Program

```
TITLE PENJ DECIMAL 4 PASANG 9 DIGIT DAN CEK OF
.MODEL SMALL
.STACK 100H

.DATA
BM1 DB 45H,34H,23H,12H,01H,00H,89H,78H,67H,56H,05H,00H,23H,22H,12H,11H,01H,00H,45H,44H,34H,33H,03H,00H
BM2 DB 67H,66H,56H,55H,05H,00H,89H,88H,78H,77H,07H,00H,01H,01H,91H,99H,09H,00H,34H,31H,21H,21H,01H,00H
BM3 DB 24 DUP(0)
BM4 DB 24 DUP(0)

;DATA 1 : 112233445H,556677889H,111122223H,333344445H
;DATA 2 : 555566667H,777788889H,999910101H,121213134H
;HASIL  : 667800112,1334466778,1111032324,454557579

;.....................................
;PROGRAMMER : ELSADANIKA MAURA SITORUS
;NIM-KELAS  : 211344013 - 2NK
;TANGGAL    : 01/02/23
;.....................................

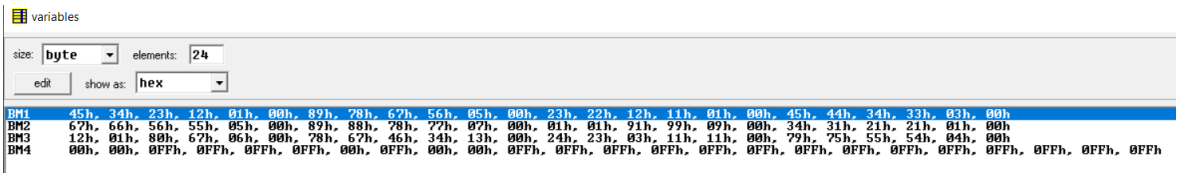
.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV SI,0
     MOV CL,0
     MOV BL,24

L1: MOV AL,[BM1+SI]
    ADC AL,[BM2+SI]
    JO L2
    MOV [BM4+SI],0FFH

L2: DAA
    MOV CL,AL
    MOV [BM3+SI],CL
    INC SI
    DEC BL
    JNZ L1

    MOV AH,4CH
    INT 21H
    MAIN ENDP
END MAIN
```

Output



F. Analisis Data

Praktikum penjumlahan data-data bilangan BCD pada intel 8086 menggunakan simulator EMU8086 yang telah dilakukan menghasilkan hasil data yang sesuai antara perhitungan teori atau manual dengan perhitungan berdasarkan program yang telah dibuat. Berikut ini adalah analisis data setiap program yang dibuat yang dibagi dalam urutan nomor soal:

1. **Menjumlahkan 10 pasang data decimal 2 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3, beri tanda overflow.**

   Data dideklarasikan dengan variabel BM1 dan BM2. Variabel BM4 dan BM3 digunakan untuk menyimpan hasil penjumlahan dengan jumlah 10 data tiap memory stack. Dikarenakan dari hasil penjumlahan data BM1 dengan BM2 akan terdapat carry, maka disediakan 1 memory stack untuk setiap hasil carry sehingga hasil penjumlahan akan dibaca berurutan dari BM3 lalu BM4. Hasil carry akan ditempatkan di BM3 supaya memudahkan pembacaan hasil jumlah data. BM5 digunakan sebagai penanda terjadinya overflow dari setiap penjumlahan data. Ketika BM5 menunjukkan hasil 0FFH artinya tidak terjadi overflow, sedangkan hasil 00H menandakan terjadinya overflow.

Pada program pertama, digunakan perintah MOV AX,@DATA ; MOV DS,AX ; dan MOV SI,0 sebagai langkah awal mengatur register untuk menginputkan data-data pada delarasi variabel awal. Selanjutnya, dilakukan penjumlahan pada label pertama (L1) untuk variabel BM1 dengan BM2 pada register AL dengan perumusan MOV AL, [BM1+SI] ; ADD AL, [BM2+SI]. Penjumlahan data pada BM1 dan BM2 dilakukan pada register AL karena akan terjadi koreksi data dengan instruksi DAA setelahnya. Koreksi DAA hanya dapat dilakukan pada register AL, sehingga register AL digunakan untuk penjumlahan BM1 dan BM2.

Carry yang terdapat dari hasil penjumlahan BM1 dan BM2 disimpan pada register AH dengan perintah ADC AH,0. Untuk memindahkan hasil data penjumlahan pada AL dan carry di AH, digunakan perintah MOV CL,AL dan MOV CH,AL sehingga data-data di-copy ke register CH:CL. Menurut permintaan soal nomor satu, diinginkan bahwa hasil penjumlahan disimpan di BM3, maka digunakan kembali perintah MOV [BM3+SI],CH dan MOV [BM4+SI], CL sehingga data-data hasil penjumlahan akan masuk ke BM3 (carry akhir) dan BM4.

Untuk menandai ketika terjadi overflow, maka digunakan perintah JO (Jump if Overflow) dan MOV [BM5+SI],0FFH. Artinya jika tidak overflow maka program akan menghasilkan 0FFH pada BM5 dan perintah JO L2 tidak akan dilaksanakan. Sebaliknya, jika terjadi overflow maka program akan menghasilkan 00H pada BM5 dan perintah JO L2 tidak terlaksana. Pada label 2, dilakukan increment SI dan CMP SI, 10 yaitu sejumlah banyaknya data. Perhitungan SI dimulai dari 0 maka jumlah SI adalah 9 sehingga program akan berhenti ketika nilai SI = 10. Lalu akan terjadi loop pada akhir perintah label 2 (L2) supaya kembali ke L1 dan melanjutkan proses penjumlahan data BM1 + BM2 pada SI 1 dan dilanjutkan dengan hal yang sama hingga data yang ke 9 dan program berhenti.

Hasil akhir data ditunjukkan pada variables, dibaca berurutan dari BM3 dan BM4. BM5 menunjukkan tanda overflow. Sedangkan BM1 dan BM2 adalah data-data yang diinginkan untuk dijumlahkan secara BCD. Hasil yang didapatkan sesuai dengan perhitungan manual/teori yaitu, 130, 126, 126, 175, 125, 71, 60, 70, 77, 87. Dengan demikian, program ini dapat dinyatakan berhasil dan sesuai dengan keinginan soal.

2. **Menjumlahkan 10 pasang data decimal 4 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3, beri tanda overflow.**

Pada program kedua, digunakan deklarasi variabel BM1 dan BM2 dengan tipe DB untuk data-data yang ingin dijumlahkan dengan penambahan data 00H yang disisipkan setiap 1 data. Hal ini dilakukan supaya hasil carry dari setiap penjumlahan data tidak mempengaruhi penjumlahan data selanjutnya. Selain itu, hal ini juga dilakukan untuk menghemat penggunaan memory stack sehingga tidak perlu menambah variabel lain untuk menempatkan carry. Data-data 4 digit bernilai 16 bit dipecah menjadi 8 bit dengan urutan low byte, high byte tiap datanya dan disisipkan 00H. Sehingga jumlah data menjadi 30 buah. BM3 digunakan untuk menyimpan

hasil penjumlahan data dengan jumlah 30 data, sedangkan BM4 digunakan untuk menyimpan tanda overflow atau tidak overflow dari setiap penjumlahan data yang telah dilakukan sejumlah 30.

Untuk badan program, setelah mengatur register untuk menginputkan data-data pada delarasi variabel awal yaitu dengan perintah MOV AX,@DATA ; MOV DS,AX ; dan MOV SI,0, digunakan perintah MOV BL,30 sebagai trigger awal untuk perintah DEC atau decrement yaitu perhitungan mundur dengan jumlah loop 30 yang akan ditempatkan pada akhir program sebelum instruksi JNZ L1. Pada label 1 (L1) digunakan perintah untuk menjumlahkan data BM1 dengan BM 2, namun pada kali ini digunakan ADC AL, [BM2+SI] dikarenakan carry akan turut dijumlahkan dan memiliki tempat untuk hasil carry tiap data yang telah disiapkan pada langkah awal yaitu deklarasi variabel data. Selanjutnya digunakan perintah JO L2 ; MOV [BM4+SI],0FFH untuk menunjukkan keberadaan overflow dari penjumlahan data BM1 dan BM2 yang telah dilakukan.

Ketika tidak terjadi overflow, maka perintah JO L2 tidak akan berlaku sehingga program akan menginputkan 0FFH ke BM4, sedangkan ketika terjadi overflow, maka perintah JO L2 berlaku sehingga program akan langsung melompat ke label 2 (L2). Pada L2 dilakukan koreksi dengan perintah DAA, setelah itu data pada register AL, dipindahkan ke CL. Data pada register CL dipindahkan kembali ke BM3. Maka penjumlahan data ke-nol telah selesai dilakukan. Selanjutnya adalah perintah INC SI, sehingga nomor data akan bertambah 1 dan dilanjutkan ke perintah DEC BL. DEC BL akan mengurangi 1 angka pada BL setiap loop. Pada program ini tidak digunakna CMP dikarenakan akan mempengaruhi nilai carry flag, sehingga CF akan selalu bernilai 1 dan menyebabkan hasil penjumlahan tidak sesuai karena bertambah 1. Jika menggunakan perintah CMP maka program harus memiliki tambahan 1 memory area atau memory stack sehingga CF tidak berpengaruh langsung ke data selanjutnya, namun langsung ditempatkan ke memory stack yang disiapkan.

Setelah perintah DEC BL, selanjutnya adalah JNZ L1 yaitu perintah untuk kembali ke L1 sehingga program akan melanjutkan penjumlahan data yang ke 1 hingga data yang ke 30. Setelah semua data selesai dijumlahkan, BL akan bernilai 0, sehingga program akan berhenti dan dianggap selesai. Hasil data ditunjukkan pada variables, BM3. Sedangkan tanda terjadinya overflow ditunjukkan pada BM4. Hasil penjumlahan yang didapatkan sangat sesuai dengan penjumlahan manual, yaitu 5130, 12883, 6444, 8203, 9475, 7185, 6010, 11423, 8127, 9223.

3. **Menjumlahkan data decimal 8 digit yang berada pada BM1 dan BM2. Simpan hasil penjumlahan pada BM3, beri tanda overflow.**

Pada program ketiga, digunakan deklarasi variabel BM1 dan BM2 dengan tipe DB untuk data-data yang ingin dijumlahkan dengan penambahan data 00H yang disisipkan setiap 1 data. Hal ini dilakukan supaya hasil carry dari setiap penjumlahan data tidak mempengaruhi penjumlahan data selanjutnya. Selain itu, hal ini juga dilakukan untuk menghemat penggunaan memory stack sehingga tidak perlu

menambah variabel lain untuk menempatkan carry. Data-data 8 digit bernilai 32 bit dipecah menjadi 8 bit dengan urutan low byte, high byte tiap datanya dan disisipkan 00H. Sehingga jumlah data menjadi 25 buah. BM3 digunakan untuk menyimpan hasil penjumlahan data dengan jumlah 25 data, sedangkan BM4 digunakan untuk menyimpan tanda overflow atau tidak overflow dari setiap penjumlahan data yang telah dilakukan sejumlah 25.

Proses yang terjadi pada program 3 adalah sama dengan program 2. Hal yang membedakan adalah jumlah loop decrement, yaitu sejumlah 25. Hal ini disesuaikan dengan jumlah data yang ada pada BM1 dan BM2 yaitu masing-masing 25 data. Setelah semua data selesai dijumlahkan, BL akan bernilai 0, sehingga program akan berhenti dan dianggap selesai. Hasil data ditunjukkan pada variables, BM3. Sedangkan tanda terjadinya overflow ditunjukkan pada BM4. Hasil penjumlahan yang didapatkan sangat sesuai dengan penjumlahan manual, yaitu 91249134, 188440871, 143322044, 12345678, 84235510.

4. **Menjumlahkan 4 pasang data decimal 9 digit. Data input dan outputnya tentukan sendiri**

Pada program keempat, digunakan deklarasi variabel BM1 dan BM2 dengan tipe DB untuk data-data yang ingin dijumlahkan dengan penambahan data 00H yang disisipkan setiap 1 data. Hal ini dilakukan supaya hasil carry dari setiap penjumlahan data tidak mempengaruhi penjumlahan data selanjutnya. Selain itu, hal ini juga dilakukan untuk menghemat penggunaan memory stack sehingga tidak perlu menambah variabel lain untuk menempatkan carry. Data-data 4 pasang 9 digit dipecah menjadi 8 bit dengan urutan low byte, high byte tiap datanya dan disisipkan 00H. Sehingga jumlah data menjadi 24 buah. BM3 digunakan untuk menyimpan hasil penjumlahan data dengan jumlah 24 data, sedangkan BM4 digunakan untuk menyimpan tanda overflow atau tidak overflow dari setiap penjumlahan data yang telah dilakukan sejumlah 24.

Data-data yang dijumlahkan adalah sebagai berikut:

Data1 : 112233445H, 556677889H, 111122223H, 333344445H

Data2 : 555566667H, 777788889H, 999910101H, 121213134H

Data 1 disimpan pada BM1 dan data 2 disimpan pada BM2 dalam bentuk 8 bit dengan sisipan 00H tiap 1 pasang data.

Proses yang terjadi pada program 4 adalah sama dengan program 2 dan 3. Hal yang membedakan adalah jumlah loop decrement, yaitu sejumlah 24. Hal ini disesuaikan dengan jumlah data yang ada pada BM1 dan BM2 yaitu masing-masing 24 data. Setelah semua data selesai dijumlahkan, BL akan bernilai 0, sehingga program akan berhenti dan dianggap selesai. Hasil data ditunjukkan pada variables, BM3. Sedangkan tanda terjadinya overflow ditunjukkan pada BM4. Hasil penjumlahan yang didapatkan sangat sesuai dengan penjumlahan manual, yaitu 667800112, 1334466778, 1111032324, 454557579 yang diurutkan dari low byte ke high byte tiap data pada BM3.

G. Kesimpulan

Melalui praktikum yang telah dilakukan, diketahui bahwa didalam penjumlahan decimal jika terdapat carry maka perlu disiapkan tempat pada memory area supaya carry tidak mempengaruhi nilai selanjutnya. Memory area yang disiapkan khusus untuk carry disisipkan pada deklarasi variabel data setelah setiap data selesai disimpan. Penambahan memory area dituliskan dalam bentuk 00H. Penambahan ini disesuaikan dengan kebutuhan memory area yang juga dilihat dari jumlah hasil carry yang dibawa dari hasil penjumlahan decimal.

Penjumlahan bilangan decimal memerlukan faktor koreksi dengan menjumlahkan biner 6 terhadap data penjumlahan jika data lebih dari 9. Hal ini dilakukan supaya tidak terjadi kekeliruan dalam hasil penjumlahan biner yang dilakukan oleh simulator. Dengan faktor koreksi ADD maka hasil menjadi sesuai dengan penjumlahan decimal yang seharusnya.

DEC BL akan mengurangi 1 angka pada BL setiap loop. Pada program ini tidak digunakna CMP dikarenakan akan mempengaruhi nilai carry flag, sehingga CF akan selalu bernilai 1 dan menyebabkan hasil penjumlahan tidak sesuai karena bertambah 1. Jika menggunakan perintah CMP maka program harus memiliki tambahan 1 memory area atau memory stack sehingga CF tidak berpengaruh langsung ke data selanjutnya, namun langsung ditempatkan ke memory stack yang disiapkan.

H. Daftar Pustaka

[1] Kho, D. Pengertian Mikroprosesor (Microprocessor) dan Cara Kerja Mikroprossesor. [teknikelektronika.com]. Available: https://teknikelektronika.com/pengertian-mikroprosesor-microprocessor-cara-kerja-mikroprosesor-adalah/

[2] Hidayatullah, S. 2021. Pengertian Mikroprosesor Beserta Fungsi dan Sejarah Mikroprosesor. [belajaronline.net]. Available: https://www.belajaronline.net/2021/01/pengertian-mikroprosesor-fungsi-dan-sejarah-mikroprosesor.html

[3] Ardiyanto88. 2013. Perintah Dasar Bahasa Assembly Menggunakan Emulator 8086. [ardiyanto88.wordpress.com]. Available: https://ardiyanto88.wordpress.com/2013/10/25/perintah-dasar-bahasa-assembly-menggunakan-emulator-8086/

I. Lampiran ASCII Code Table

ASCII Table

The ASCII code is used to give to each symbol / key from the keyboard a unique number called ASCII code.
It can be used to convert text into ASCII code and then into binary code. It can be used within your code to identify specific characters in a string or specific keys being pressed on the keyboard.

The ASCII table contains 256 codes (from 0 to 255). The table below only shows the most useful ASCII codes.

Char	ASCII Code (Decimal)
a	97
b	98
c	99
d	100
e	101
f	102
g	103
h	104
i	105
j	106
k	107
l	108
m	109
n	110
o	111
p	112
q	113
r	114
s	115
t	116
u	117
v	118
w	119
x	120
y	121
z	122

Char	ASCII Code (Decimal)
A	65
B	66
C	67
D	68
E	69
F	70
G	71
H	72
I	73
J	74
K	75
L	76
M	77
N	78
O	79
P	80
Q	81
R	82
S	83
T	84
U	85
V	86
W	87
X	88
Y	89
Z	90

Char	ASCII Code (Decimal)
space	32
!	33
"	34
#	35
$	36
%	37
&	38
'	39
(	40
)	41
*	42
+	43
,	44
-	45
.	46
/	47
:	58
;	59
<	60
=	61
>	62
?	63
@	64
[	91
\	92
]	93
^	94
_	95
`	96
{	123
|	124
}	125
~	126
‘	145
’	146
“	147
”	148
•	149
˜	152

Char	ASCII Code (Decimal)
0	48
1	49
2	50
3	51
4	52
5	53
6	54
7	55
8	56
9	57

Char	ASCII Code (Decimal)
€	128
£	163
¥	165
$	36
©	169
™	153
°	176
˜	152
¡	161
¿	191

© 101 Computing – www.101Computing.net